BACKEND DEVELOPMENT — DAY 2

FastAPI Routes, Path Parameters, Query Parameters, and Request Bodies

Beginner notes with working code examples and simple explanations

Your starting point: You know Python fundamentals, but you are new to web frameworks and databases. These notes start from that level and use a Hospital ERP example throughout.

Today's goal: make your API respond to different URLs, accept input, check that input, and return useful JSON.

1. What you will learn today

In Day 1, you learned that an API lets another program communicate with your Python application. Today, you will learn how to make different API endpoints.

By the end, you should be able to:

• Create more than one route in a FastAPI application.

• Read an ID from a URL using a path parameter.

• Read filters from a URL using query parameters.

• Receive JSON data using a request body and a Pydantic model.

• Understand how FastAPI validates input and reports mistakes.

• Test your API in the automatic /docs page.

A small dictionary

Word	Simple meaning	Hospital ERP example
Route / endpoint	A URL path your app knows how to handle	/doctors
Path parameter	A value that is part of the path	/doctors/12
Query parameter	An optional setting after ?	/doctors?branch=Cardiology
Request body	Data sent inside a request, usually JSON	New patient's name and age
Validation	Checking that input follows rules	Age must be a number

2. Start with a working FastAPI file

Open your Day 1 project in VS Code. Keep your virtual environment from Day 1 if you already created one. Create or open **main.py**.

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def home():
    return {"message": "Hospital ERP API is running"}
```

Read each part

• **from fastapi import FastAPI** — imports the tool used to create the API.

• **app = FastAPI()** — creates your application object.

• **@app.get("/")** — tells FastAPI: when someone sends a GET request to the home path /, run the function below.

• **def home():** — an ordinary Python function that handles the request.

• **return { ... }** — returns a Python dictionary; FastAPI converts it into a JSON response.

> **Run it:** in the VS Code terminal, activate your virtual environment if needed, then use the development command shown in the current FastAPI tutorial. In a typical FastAPI installation, `fastapi dev main.py` starts the local development server.

Open http://127.0.0.1:8000/ to see the response, then open http://127.0.0.1:8000/docs to test endpoints. The official tutorial is the source of truth if your installed version's command differs.

3. Routes: give each task its own URL

Think of routes as different service counters in a hospital. One counter lists doctors; another shows one doctor; another registers a patient. Each route connects a URL and an HTTP method to a Python function.

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/")
def home():
    return {"message": "Hospital ERP API is running"}


@app.get("/doctors")
def list_doctors():
    return {"doctors": ["Dr. Rao", "Dr. Meena"]}


@app.get("/patients")
def list_patients():
    return {"patients": ["Patient A", "Patient B"]}
```

Try these URLs in your browser or in /docs:

- `http://127.0.0.1:8000/` — home message
- `http://127.0.0.1:8000/doctors` — doctor list
- `http://127.0.0.1:8000/patients` — patient list

> **Important:** these lists are temporary examples written directly in Python. They are not saved in a database, and they are not connected to your existing JSON files yet.

4. Path parameters: get one record by its ID

A path parameter is a value placed inside the URL path. For example, in `/doctors/12`, the number 12 can represent the doctor's ID.

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/doctors/{doctor_id}")
def get_doctor(doctor_id: int):
    return {
        "doctor_id": doctor_id,
        "message": "Here is the requested doctor"
    }
```

Understand the important line

`@app.get("/doctors/{doctor_id}")` means the value in that part of the URL will be passed to the function. The function parameter name `doctor_id` matches the name inside the curly braces.

`doctor_id: int` tells FastAPI that this value should be an integer. FastAPI converts a valid number from the URL into a Python integer and rejects invalid input.

Try it

- `/doctors/12` → valid; the response includes doctor_id 12.
- `/doctors/abc` → invalid; FastAPI returns a validation error instead of calling your function normally.

> This is useful for your Hospital ERP because a client can ask for one specific doctor or patient by ID. In a real app, you would next look up that ID in a database and return the matching record.

5. Query parameters: filter or customize a list

Query parameters appear after a question mark ? in the URL. They are good for filters, search text, sorting, and pagination.

```
from fastapi import FastAPI

app = FastAPI()


@app.get("/doctors")
def list_doctors(branch: str | None = None, limit: int = 10):
    return {
        "branch_filter": branch,
        "limit": limit
    }
```

Try these URLs

- `/doctors` → branch is None and limit is 10.
- `/doctors?branch=Cardiology` → branch is Cardiology and limit is 10.
- `/doctors?branch=Cardiology&limit=5` → branch is Cardiology and limit is 5.

Why is one optional? `branch: str | None = None` means branch can be text or None, and its default is None. `limit: int = 10` means the user may provide a number, otherwise it defaults to 10.

> FastAPI reads the values from the URL and validates their types. For example, `?limit=hello` produces a validation error because limit should be an integer.

6. Path parameter vs query parameter

Question	Path parameter	Query parameter
Example	/doctors/12	/doctors?branch=Cardiology
Usually used for	Identify one resource	Filter or customize results
Required?	Usually required when in the route path	Can be optional with a default
FastAPI example	doctor_id: int	branch: str | None = None

7. Request body: send new data to the API

When a user creates a patient, the client must send information such as name and age. That information is usually sent in the request body as JSON. FastAPI uses a Pydantic model to describe and validate the expected shape of that data.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class PatientCreate(BaseModel):
    name: str
    age: int
    department: str


@app.post("/patients")
def create_patient(patient: PatientCreate):
    return {
        "message": "Patient details received",
        "patient": patient.model_dump()
    }
```

What is each part doing?

• **BaseModel** — a Pydantic base class used to define the data fields an API expects.

• **class PatientCreate(BaseModel)** — describes the shape of incoming patient data.

• **name: str** — name must be text.

• **age: int** — age must be an integer.

• **@app.post("/patients")** — defines an endpoint commonly used to create a resource.

• **patient: PatientCreate** — tells FastAPI to read JSON from the request body and validate it against this model.

• **patient.model_dump()** — turns the validated model into a regular Python dictionary.

Example JSON to send

```
{
    "name": "Ravi",
    "age": 35,
    "department": "Cardiology"
}
```

Open `/docs`, expand `POST /patients`, choose **Try it out**, enter the JSON example, and execute it. FastAPI should return the data it received.

> If age is sent as text that cannot be converted to an integer, or if a required field is missing, FastAPI returns a validation error (commonly HTTP 422).

8. What is Pydantic? Is it a database?

No. Pydantic is for describing and validating data. A database is for storing data so it can be retrieved later. These are different jobs.

Tool	Job	Hospital example
FastAPI	Receives HTTP requests and sends responses	POST /patients
Pydantic	Checks the incoming data shape and types	Checks name is text and age is an integer

Tool	Job	Hospital example
Database	Stores and retrieves records persistently	Patient remains after server restart

In today's examples, the response only echoes the patient data back. It does not save the patient. Database learning comes later.

9. Combine path, query, and body

FastAPI can use these three types of input in one endpoint. This example updates a particular patient's details and optionally accepts a note in the URL.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class PatientUpdate(BaseModel):
    name: str
    age: int


@app.put("/patients/{patient_id}")
def update_patient(
    patient_id: int,
    patient: PatientUpdate,
    note: str | None = None
):
    return {
        "patient_id": patient_id,
        "updated_patient": patient.model_dump(),
        "note": note
    }
```

Example request: `PUT /patients/42?note=corrected-age` with this JSON body:

```
{
    "name": "Ravi Kumar",
    "age": 36
}
```

The response combines patient_id from the path, note from the query string, and name/age from the JSON body. In a real app, the function would also check that patient 42 exists, apply authorization rules, and update a database.

10. Test your work in VS Code

For each code example, use one example at a time in `main.py`. Stop and restart the development server after making changes if your server does not reload automatically.

■ Open the project folder in VS Code.

■ Activate the virtual environment you created on Day 1.

■ Make sure FastAPI is installed in that environment.

■ Put one complete example into main.py and save the file.

■ Start the development server using the FastAPI command from Day 1.

■ Open http://127.0.0.1:8000/docs.

■ Test each endpoint with normal input and invalid input.

■ Read the response and the HTTP status code.

Common beginner mistakes

• **Using /doctor/12 but defining /doctors/{doctor_id}:** the path must match exactly.

• **Writing doctor_id: int but testing /doctors/abc:** abc is not an integer, so a validation error is expected.

• **Forgetting a required JSON field:** send all required fields from the Pydantic model.

• **Thinking the response means data was saved:** returning data is not the same as storing it.

• **Using a GET request to create a patient:** use POST for the typical create operation.

11. Practice tasks — do these yourself

Do not just copy the examples. Change them and observe what happens.

• **Task 1:** Create GET /hospital that returns the hospital name.

• **Task 2:** Create GET /doctors/{doctor_id} that accepts an integer ID and returns it.

• **Task 3:** Add a query parameter branch to GET /doctors, with a default of None.

• **Task 4:** Create a Pydantic model for a doctor with name (text), experience_years (integer), and branch (text).

• **Task 5:** Create POST /doctors that accepts your model and returns the validated data.

• **Task 6:** Test a valid request, a missing field, a text value where an integer is expected, and a URL with the wrong path.

• **Task 7:** Explain in your own words why Pydantic is not a database.

12. Quick self-test

1. What is a route?

Answer: A URL path and HTTP method connected to a function.

2. In /patients/25, what is 25?

Answer: A path parameter value.

3. In /doctors?branch=ENT, what is branch?

Answer: A query parameter.

4. What is a request body used for?

Answer: Sending data such as new patient details, usually as JSON.

5. What does BaseModel help with?

Answer: Describing and validating the structure of incoming data.

6. Does returning patient data automatically save it?

Answer: No. Persistent saving needs storage such as a database or file.

7. Why does /doctors/abc fail if doctor_id is int?

Answer: FastAPI cannot validate abc as an integer.

13. Day 2 completion checklist

■ I can explain a route in simple language.

■ I can tell path parameters and query parameters apart.

■ I can explain why type hints like int matter in FastAPI.

■ I can accept JSON with a Pydantic model.

■ I can test success and validation errors using /docs.

■ I understand that this example does not yet use a database.

Next lesson: Day 3 — stronger validation, optional fields, handling errors, and returning the right HTTP status codes. After that, we can start organizing the Hospital ERP API into separate files.

Official references

These notes use the concepts and examples covered by the official FastAPI documentation. Use these pages when you want to read the original material:

- FastAPI — First Steps
- FastAPI — Path Parameters
- FastAPI — Query Parameters
- FastAPI — Request Body
- FastAPI — Tutorial index